

Full Stack Developer — Step 1

HTML & CSS

Roadmap objective: Build accessible, semantic and responsive frontend foundations

Level: Beginner → Intermediate

Last reviewed: September 2026

How to use these notes

Read the concepts first, reproduce the examples yourself, complete the practical lab, and answer the interview questions without looking at the notes. Use the official documentation links as the final authority for changing APIs and platform behavior.

Learning outcomes

- Semantic HTML
- Forms
- Accessibility basics
- CSS selectors
- Box model
- Flexbox
- Grid
- Responsive design
- Layout debugging

Core concepts

1. Semantic HTML communicates document structure to browsers, assistive technology and developers.
2. CSS controls presentation; the box model explains content, padding, border and margin.
3. Flexbox is generally useful for one-dimensional alignment; Grid is powerful for two-dimensional page layouts.
4. Responsive design should adapt layout and content to viewport constraints rather than target one device model.

Concept map

```
HTML & CSS
|
+--> Learn the concept
|
+--> Reproduce a small example
|
+--> Apply it in a feature
|
+--> Test failure/edge cases
|
+--> Document the decision
```

Detailed study guide

1. Semantic HTML

Study semantic html through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

2. Forms

Study forms through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

3. Accessibility basics

Study accessibility basics through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

4. CSS selectors

Study css selectors through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

5. Box model

Study box model through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

6. Flexbox

Study flexbox through three layers: mental model, implementation and practical trade-offs. First explain what it is and what problem it solves. Then reproduce a minimal example without copying. Finally, decide where it belongs in a real application and what can go wrong. Test empty, invalid, boundary and repeated inputs. If the concept interacts with another layer, identify the boundary explicitly so that failures can be isolated instead of guessed at.

Worked example

Semantic HTML + responsive CSS

```
<main>
  <h1>Career Roadmap</h1>
  <section aria-labelledby="step-title">
    <h2 id="step-title">Step 1</h2>
    <p>Learn the foundations.</p>
  </section>
</main>

.container {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 1rem;
}
```

```
@media (max-width: 700px) {  
  .container { grid-template-columns: 1fr; }  
}
```

Practice variation: change one input, introduce one failure, predict the result, then run it. Rewrite the example from memory afterward.

Comparison table

Concept	Meaning	Practical use
Flexbox	One-dimensional	Row/column alignment
Grid	Two-dimensional	Rows + columns
Semantic HTML	Meaning	Structure/accessibility

Common mistakes

- Div soup
- Fixed widths everywhere
- Missing form labels
- Ignoring keyboard navigation
- Using CSS hacks instead of understanding layout

Best practices

- Keep responsibilities clear and small.
- Validate inputs at system boundaries.
- Prefer readable code over clever code.
- Test important success and failure paths.
- Use official documentation for current API/platform behavior.
- Keep secrets and credentials out of source control.
- Document important trade-offs so another developer can reproduce your reasoning.

Extended practical lab

Complete these tasks in order. The goal is to turn passive knowledge into evidence that you can actually use the topic.

Lab 1

- Build the page with semantic HTML first.
- Write down what you expected, what happened, and why.

Lab 2

- Add labels and keyboard-friendly controls.
- Write down what you expected, what happened, and why.

Lab 3

- Use Flexbox for one-dimensional layout.
- Write down what you expected, what happened, and why.

Lab 4

- Use Grid for a two-dimensional section.
- Write down what you expected, what happened, and why.

Lab 5

- Add a mobile breakpoint.
- Write down what you expected, what happened, and why.

Lab 6

- Inspect the page with browser accessibility/dev tools.
- Write down what you expected, what happened, and why.

Mini-project

Build a responsive career landing page using semantic HTML, Flexbox/Grid, forms and accessible labels. Test keyboard navigation and multiple viewport widths.

Acceptance criteria

- A new developer can understand the project from its README.
- The main happy path works from start to finish.
- At least three edge/failure cases are tested.
- The project has meaningful Git commits.
- The project includes a short explanation of design choices.
- If deployment applies, production behavior is verified rather than assumed.

Interview preparation

Practice answering these aloud. A strong answer should define the concept, give a concrete example, mention one trade-off, and explain when you would use it.

Why semantic HTML?

- Answer structure: definition → example → trade-off/limitation → practical use.

Flexbox vs Grid?

- Answer structure: definition → example → trade-off/limitation → practical use.

What is the CSS box model?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do you make a form accessible?

- Answer structure: definition → example → trade-off/limitation → practical use.

How do responsive layouts work?

- Answer structure: definition → example → trade-off/limitation → practical use.

Why avoid layout hacks?

- Answer structure: definition → example → trade-off/limitation → practical use.

Debugging checklist

- Reproduce the problem consistently.
- Reduce it to the smallest failing example.
- Inspect inputs at each boundary.
- Check the first incorrect value, not only the final error.
- Read the complete error message and stack trace.
- Change one thing at a time.
- Retest the original failure after the fix.
- Add a regression test when the bug is important.

Glossary

Term	Your own definition	Example/use
Semantic HTML	Write this in your own words.	Give one project example.
DOM	Write this in your own words.	Give one project example.
Box model	Write this in your own words.	Give one project example.
Flexbox	Write this in your own words.	Give one project example.
Grid	Write this in your own words.	Give one project example.
Breakpoint	Write this in your own words.	Give one project example.
Responsive design	Write this in your own words.	Give one project example.
Accessibility	Write this in your own words.	Give one project example.

Term	Your own definition	Example/use
Focus	Write this in your own words.	Give one project example.
Form label	Write this in your own words.	Give one project example.

Self-test

- Explain Semantic HTML without using its textbook definition.
- Show a small example of Forms.
- What is the most important boundary in this topic?
- What is one failure mode and how would you diagnose it?
- What trade-off should a developer know before using this technique?
- How would you test the normal path and an edge case?
- How does this topic connect to the next roadmap step?
- What would you change if the system had 100× more users/data?
- What part of this topic would you explain differently to a beginner?
- What can you now build that you could not build before studying this step?

Final checklist

- I can explain and demonstrate Semantic HTML.
- I can explain and demonstrate Forms.
- I can explain and demonstrate Accessibility basics.
- I can explain and demonstrate CSS selectors.
- I can explain and demonstrate Box model.
- I can explain and demonstrate Flexbox.
- I can explain and demonstrate Grid.
- I can explain and demonstrate Responsive design.
- I can explain and demonstrate Layout debugging.
- I completed the practical lab.
- I built or extended the mini-project.
- I tested at least three failure/edge cases.
- I can explain one trade-off.
- I can answer the interview questions without notes.
- I know where to find the authoritative documentation.

Recommended documentation

- <https://developer.mozilla.org/>
- <https://www.w3.org/TR/WCAG22/>

Recommended videos

- Apna College HTML (Existing DB resource 211)
- Apna College CSS (Existing DB resource 215)

Source note: resource references are based on the existing CareerCompass database plus verified web research. For changing APIs, versions, deployment settings and security guidance, consult the linked official documentation before production use.